Лабораторная работа № 5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP.

Выполнила: Колодезникова Софья, 11-400

Параметры эксперимента

Данные генерируются программно с фиксированным seed = 12345: признаки равномерно распределены в [−1, 1], целевое значение y = Xw_true + 0,5 + шум, шум в [−0,1; 0,1], w_true разрежён. Начальные значения w = 0, b = 0, шаг обучения $\eta = 0{,}5$, число итераций K = 100, $\varepsilon = 10^{-6}$. Базовые параметры: $N = 10^5$, d = 32, $\lambda = 10^{-3}$, 8 потоков; в экспериментах 3–6 меняется только исследуемый параметр.

Время измеряется функцией omp_get_wtime() только для обучения. Для каждой конфигурации берётся медиана 5 замеров. Корректность: $\Delta w < 10^{-8}$ и $|\Delta F| < 10^{-10}$ относительно последовательной версии. Компиляция: gcc -O3 -march=native -fopenmp.

Характеристики компьютера

Модель процессора: Core Ultra 5 125H
Число ядер: 14 (18 логических процессоров)
Размеры кешей: L1 = 1,4 МБ, L2 = 14 МБ, L3 = 18 МБ
Версия компилятора: GCC 16.2.0

Результаты экспериментов

Эксперимент 1. Проверка корректности.

Реализация	Итоговая функция потерь	Δ_w	Корректность подтверждена?
Последовательная	0.0156496535	0	Да
OpenMP	0.0156496535	0	Да
SIMD	0.0156496535	$2.2 * 10^{-16}$	Да
OpenMP + SIMD	0.0156496535	$2.2 * 10^{-16}$	Да

Итог. Допуск: $\Delta w < 10^{-8}$ и $|\Delta F| < 10^{-10}$ относительно последовательной версии. Итоговая функция потерь у всех вариантов одинакова. Отличие вызвано другим порядком суммирования при векторизации, поэтому результаты можно считать совпадающими.

Эксперимент 2. Сравнение реализаций.

Вариант	Время, с	Ускорение S
Последовательная	0.2300	1.00
OpenMP	0.0825	2.79
SIMD	0.2340	0.98
OpenMP + SIMD	0.0955	2.41

Итог. При $N = 10^5$, d = 32, K = 100 и 8 потоках OpenMP даёт ускорение 2,79. SIMD не даёт выигрыша, а их сочетание немного медленнее, чем OpenMP.

Эксперимент 3. Влияние числа объектов N.

N	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP+SIMD, с
10^4	0.0104	0.0074	0.0077	0.0136
10^5	0.2420	0.0690	0.2420	0.1165
10^6	2.0300	0.7460	1.8390	0.7260

Итог. При $N = 10^4$ OpenMP даёт лишь 1,4x, а OpenMP + SIMD медленнее последовательной версии. Работы мало, и на каждой из K итераций тратится время на создание параллельной области, барьер и слияние частичных градиентов. При $N = 10^5$ эти расходы окупаются: OpenMP даёт 3,5x. При $N = 10^6$ ускорение OpenMP снижается до 2,7x, поскольку все потоки делят пропускную способность памяти. Время OpenMP + SIMD при этом сравнивается с OpenMP, а SIMD даёт только 1,10x.

Эксперимент 4. Влияние числа признаков d.

d	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP+SIMD, с
8	0.0371	0.0196	0.0461	0.0230
32	0.2235	0.0745	0.2150	0.0905
128	1.1250	0.3290	0.6790	0.2920
256	2.9610	0.7300	2.3510	0.7150

Итог. Относительная эффективность SIMD растёт с d. Причины:
- при малом d внутренние циклы слишком короткие, и накладные расходы съедают выигрыш;
- при росте d цикл по признакам становится длиннее, векторные инструкции используются полнее;
- при d = 256 матрица занимает около 205 МБ, задача упирается в память, выигрыш SIMD снова падает.

Эксперимент 5. Влияние числа потоков.

Потоки p	Время OpenMP, с	S_p OpenMP	E_p OpenMP	Время OpenMP+SIMD, с	S_p OpenMP+SIMD	E_p OpenMP+SIMD
1	0.2170	1.00	1.00	0.2100	1.00	1.00
2	0.1635	1.33	0.66	0.1670	1.26	0.63
4	0.1205	1.80	0.45	0.1080	1.94	0.49
8	0.0905	2.40	0.30	0.1100	1.91	0.24
16	0.0780	2.78	0.17	0.0890	2.36	0.15

Итог. При базовых параметрах последовательная версия работает 0,230 с, OpenMP даёт ускорение 2,79, SIMD — 0,98, OpenMP + SIMD — 2,41. С ростом числа потоков ускорение OpenMP растёт медленно. У OpenMP + SIMD оно почти такое же. Эффективность при этом падает с 0,66 до 0,17 у OpenMP и с 0,63 до 0,15 у OpenMP + SIMD.

Эксперимент 6. Влияние коэффициента регуляризации λ.

λ	Итоговая функция потерь F(w, b)	Число коэффициентов, для которых $\|w_j\| < \varepsilon$	Время обучения, с
0	0.0016631118	0	0.1100
10^{-4}	0.0030630781	16	0.1005
10^{-3}	0.0156496535	24	0.1060
10^{-2}	0.1404431404	24	0.1145
10^{-1}	1.2811678538	24	0.0965

Итог. С ростом λ всё больше коэффициентов обнуляется. Значения 24 соответствуют всем нулевым коэффициентам в w_true, то есть модель восстанавливает разреженную структуру. Функция потерь растёт, потому что в F входит слагаемое $\lambda \cdot \Sigma|w_j|$, а ненулевые коэффициенты ещё и немного сжимаются.

Эксперимент 7. Влияние оптимизаций компилятора и автоматическая векторизация.

Флаги компиляции	Время, с	Ускорение относительно первой строки	Примечания
-O0 -fopenmp	0.3385	1.00	
-O3 -fopenmp	0.0980	3.45	Совпадают в рамках шума
-O3 -march=native -fopenmp	0.0990	3.42	
-O3 -march=native -fopenmp + omp simd	0.1070	3.16	Не дал выигрыша

Итог:

- Оптимизация -O3 ускоряет программу в 3,45x относительно -O0.

- Флаг -march=native результат не меняет, как и omp simd, так как внутренние циклы GCC уже векторизует при -O3, а время ограничивает память.

- Не удалось векторизовать цикл в update с soft-threshold, так как внутри ветвления.

- Не векторизованы внешние циклы по итерациям и объектам и цикл по потокам при слиянии градиентов.

Вывод

Корректность подтверждена для всех вариантов: $\Delta w \leq 2{,}2 \cdot 10^{-16}$, функции потерь совпадают. Различия вызваны порядком суммирования.

1. На малых и средних размерах быстрее всех OpenMP. При $N = 10^6$ и $d \geq 128$ OpenMP + SIMD сравнивается с OpenMP или чуть обгоняет его. Лучшее ускорение получилось при d = 256. Одиночный SIMD быстрее последовательной версии только при $d \geq 128$.
2. При базовых параметрах ускорение OpenMP составило 2,79, OpenMP + SIMD — 2,41, при больших N и d они почти совпадают. Ускорение меньше числа потоков по нескольким причинам:
   a. задача упирается в пропускную способность памяти;
   b. слияние частичных градиентов и обновление весов выполняются последовательно;
   c. на каждой итерации тратится время на параллельную область и барьер;
   d. потоки сверх числа производительных ядер работают на менее быстрых E-ядрах.
3. Эффективность падает с ростом числа потоков. Ускорение насыщается: при переходе от 8 к 16 потокам оно выросло лишь с 2,40 до 2,78. Это говорит об упоре в память и росте накладных расходов. Малое N тоже даёт большие накладные расходы.
4. По отчёту GCC в списке неудач нет внутренних циклов по j, то есть они векторизованы автоматически. Не векторизованы:
   a. цикл порога в update из-за ветвления;
   b. внешние циклы по итерациям и объектам;
   c. цикл по потокам при слиянии градиентов.

   Добавление omp simd почти ничего не изменило. Эффект виден только при $d \geq 128$, когда циклы достаточно длинные.
5. С ростом λ растёт число нулевых коэффициентов. Это все нулевые коэффициенты в w_true, то есть восстановлена разрежённая структура модели. Функция потерь растёт, потому что увеличивается штраф, а ненулевые коэффициенты слегка сжимаются. На время обучения λ не влияет.
6. Главный ограничитель производительности — доступ к памяти: матрица X читается целиком на каждой итерации, поэтому ускорение вычислений почти не помогает, а OpenMP упирается в общую пропускную способность. При малом N важны накладные расходы потоков и слияние частичных градиентов.